Homework 3: Robot Manipulation and Semantic Robot Knowledge

Human-centered Intelligent System Lab
National Yang Ming Chiao Tung University

Fall 2026

TA / Tutor Teaching Edition

Introduction

In this homework you will connect symbolic robot knowledge to the mathematics and execution of manipulation. You first represent robot specifications and FK/IK executions as structured RDF and validate them with SHACL. You then implement forward and inverse kinematics for a simulated six-degree-of-freedom UR5 arm. Finally, your solvers drive a deterministic finite-state-machine (FSM) pick-and-place pipeline in PyBullet [1].

The semantic layer deliberately comes first. Task 1 uses executions produced by the TA reference solvers, so it does not depend on your FK or IK code. Once your solvers exist, the same grounding and validation layer gives immediate, machine-readable feedback on their executions.

TA/Tutor Note.

Teaching introduction. Present HW3 as one continuous information loop, not four unrelated exercises:

robot specification → FK/IK execution → RDF facts → SHACL diagnosis → FSM behavior.

Task 1 defines what a valid execution record means. Tasks 2 and 3 produce the kinematic results stored in those records. Task 4 shows the operational consequence of errors: a numerical discrepancy becomes a named failure at a specific manipulation state.
At the beginning of a tutorial, emphasize three boundaries:

- RDF grounding records observations; it does not prove that the kinematics are correct.
- SHACL checks explicit data against closed-world constraints; it is not an IK solver or an OWL reasoner.
- Passing isolated FK/IK tests is necessary but the FSM additionally tests consistency under repeated closed-loop execution.

Students should be able to trace one failure from its numerical source, through its semantic flag, to the FSM state in which it matters.

1 Learning Objectives

After completing the assignment, you should be able to:

1. ground robot specifications and kinematic executions as structured RDF using CORA, SOMA, IEEE 1872 POS, and QUDT vocabularies;
2. author SHACL constraints, including a SHACL–SPARQL comparison across nodes, and interpret validation reports [4];
3. compose classic Denavit–Hartenberg transforms and construct a 6×6 geometric Jacobian [2];
4. implement and tune iterative Jacobian-based inverse kinematics; and
5. diagnose kinematic failures within a complete manipulation FSM.

2 Assignment at a Glance

Task	Work	Files you edit	Points
1	Semantic grounding and SHACL validation	`semantic/ground_execution.py`, `semantic/shapes.ttl`	30
2	Forward kinematics and Jacobian	`fk.py`	20
3	Iterative inverse kinematics	`ik.py`	40
4	FSM manipulation integration (no code changes)	None	10
	Total		**100**

Students modify exactly the four files listed above. Treat all other files as provided infrastructure.

3 Environment Setup

The verified environment is Linux with Python 3.7, PyBullet, NumPy, SciPy, and JDK 11 or newer. A GPU is not used.

```
cd hw3_template
conda create --name taica-hw3 python=3.7 -y
conda activate taica-hw3
pip install pybullet numpy scipy
sudo apt-get -y install openjdk-11-jdk
java -version && javac -version
```

Task 1 downloads Apache Jena 4.10.0 (approximately 30 MB) on its first run and caches it under `semantic/.cache/`. Network access is not needed after the cache and Python environment are prepared. No dataset or learned model is downloaded.

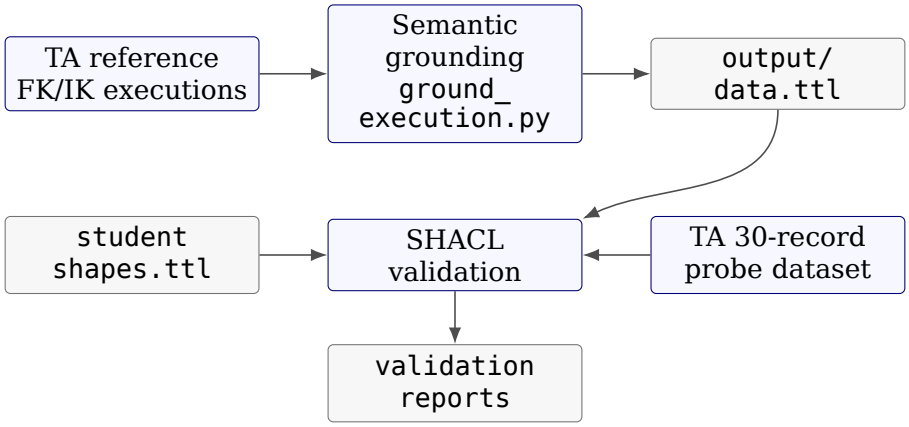
TA/Tutor Note.

The release was verified on Ubuntu 20.04 or newer. macOS and Windows are not officially supported. On lab machines whose existing environment is named `physical-ai-hw3`, a TA may prefix the semantic command with `PYTHON=~/miniconda3/envs/physical-ai-hw3/bin/python`.

4 Task 1: Semantic Grounding and SHACL Validation (30 points)

4.1 Execution model and workflow

Convert numerical FK/IK execution records into a semantic graph, then use SHACL to detect problem states directly from the stored numbers:



The design follows the REUSE principle: a robot is a `cora:Robot`; joints and joint states use SOMA; poses use `soma:6DPose` and IEEE 1872 POS; units use QUDT IRIs. The local `hw3:` vocabulary is only for concepts not covered by those standards. Numeric values must be structured nodes and typed `xsd:double` literals, never JSON strings.

4.2 Mathematical representation

Let an RDF data graph be a finite set of triples

$$G \subseteq \mathcal{N} \times \mathcal{P} \times (\mathcal{N} \cup \mathcal{L}),$$

where $\mathcal{N}$, $\mathcal{P}$, and $\mathcal{L}$ are the sets of nodes, properties, and literals. We use the predicate notation

$$C(x) \iff (x, \mathsf{rdf{:}type}, C) \in G, \qquad p(x, y) \iff (x, p, y) \in G.$$

Grounding is a serialization function

$$\gamma : \mathcal{E} \longrightarrow 2^{G}, \qquad G_{\text{exec}} = G_{\text{ontology}} \cup \bigcup_{e \in \mathcal{E}} \gamma(e),$$

which maps each robot specification or execution record e to RDF triples. It must preserve structure: a six-joint configuration c satisfies

$$\mathsf{JointConfiguration}(c) \ \wedge \ |\{s : \mathsf{hasJointState}(c, s)\}| = 6,$$

and every linked state carries its joint identity and numerical angle:

$$\forall s \, [\mathsf{hasJointState}(c, s) \Rightarrow \exists j, v \ (\mathsf{isStateOfJoint}(s, j) \wedge \mathsf{hasJointPosition}(s, v))].$$

A grounded IK execution x similarly connects one robot, target, output configuration, status, residual r_x, and target distance d_x.

For shapes graph S and data graph G, define validation as

$$\mathrm{Report}(S, G) = \{(x, m, p, v) : x \text{ violates a constraint with message } m \text{ on path } p \text{ at value } v\}.$$

The graph conforms exactly when

$$\mathrm{conforms}(S, G) \iff \mathrm{Report}(S, G) = \emptyset.$$

The four numerical course constraints are

$$\begin{aligned}
\Phi_{\mathrm{FK}}: &\quad e_x \le 0.005, && x : \mathsf{FKComputation},\\
\Phi_{\mathrm{AR}}: &\quad d_x \le 0.90, && x : \mathsf{IKComputation},\\
\Phi_{\mathrm{NC}}: &\quad r_x \le 0.02, && x : \mathsf{IKComputation},\\
\Phi_{\mathrm{JL}}: &\quad \ell_j \le q_s \le u_j, && s : \mathsf{JointState},\ \mathsf{isStateOfJoint}(s, j).
\end{aligned}$$

Violation produces, respectively, `FK_INACCURATE`, `ARM_OUT_OF_RANGE`, `NO_CONVERGENCE`, or `JOINT_LIMIT_VIOLATION`. All inequalities are inclusive: equality with a threshold or joint bound conforms.

TA/Tutor Note.

Formal reading for teaching. The grounding function γ does not infer a solution; it preserves the structure and values of an execution in the graph. SHACL evaluates predicates over the explicit extension of G. This is a closed-world validation reading: a missing triple is not treated as an unknown fact that a reasoner may later supply. The violation sets can be written directly as

$$\begin{aligned}
V_{\mathrm{AR}} &= \{x : \mathsf{IKComputation}(x) \wedge d_x > 0.90\},\\
V_{\mathrm{NC}} &= \{x : \mathsf{IKComputation}(x) \wedge r_x > 0.02\},\\
V_{\mathrm{JL}} &= \{s : \exists j, q_s, \ell_j, u_j\ (\mathsf{isStateOfJoint}(s, j) \wedge (q_s < \ell_j \vee q_s > u_j))\}.
\end{aligned}$$

Thus `sh:maxInclusive` implements the first two conformance inequalities, while the SPARQL `FILTER` implements the negation of Φ_{JL}.
Stress the distinction between structural validity and numerical conformance. A record may have all required typed nodes and still violate a numeric shape; conversely, a missing property can satisfy a range check vacuously unless a separate `sh:minCount` structure shape requires its presence.

4.3 Grounding requirements

Read the worked example `fk_computation_to_triples()` and the serializers in `semantic/common.py`. Implement:

1. `robot_spec_to_triples()`: create `stu:my_ur5` as a `cora:Robot`, record its producer and six DoF, and link six `soma:RevoluteJoint` instances. Each joint must contain its index, D–H parameters a, d, α, and inclusive lower/upper limits.
2. `ik_computation_to_triples()`: create one `hw3:IKComputation` with provenance, robot, target, status, residual, target distance, and a structured six-state joint configuration. Use `common.joint_config_to_ttl()`.

TA/Tutor Note.

Reference grounding pattern: emit one robot node, then loop over `zip(dh_params, joint_limits)` to emit six joints. Round numeric values and explicitly serialize them as typed `xsd:double` literals. For IK, use the node `stu:ik_<target>` and append the output of `joint_config_to_ttl()`. Typical S1 failures are missing types, untyped decimals, or a configuration with other than six joint states.
When explaining the graph, select one IK record and follow each edge aloud: the computation was produced by a student/group, computed for one robot, solves one target, reports status and residual, and links to a configuration; that configuration then links to six joint-state nodes. This makes the difference between a string containing six numbers and a queryable structured representation concrete.

4.4 SHACL requirements

The provided FK accuracy shape is a worked example: pose error must be at most 0.005 m. Add the following three shapes:

Condition	Legal range	Required message prefix
Target distance	$d \leq 0.90$ m	`ARM_OUT_OF_RANGE:`
IK residual	$r \leq 0.02$ m	`NO_CONVERGENCE:`
Joint position	$q_{\min} \leq q \leq q_{\max}$	`JOINT_LIMIT_VIOLATION:`

Use `sh:maxInclusive` for the first two constraints. The third must be a SHACL–SPARQL constraint on `soma:JointState`, because its position and the linked joint's bounds occur on different nodes. Its filter must use strict < and > comparisons; equality with a bound is legal.

TA/Tutor Note.

The reference suite uses node shapes targeting `hw3:IKComputation` for distance/residual. The joint-limit query obtains `?v`, `?lo`, and `?hi` through `hw3:isStateOfJoint`, then applies `FILTER(?v<?lo||?v>?hi)`. The exact prefix of each `sh:message` is grading-significant.
Explain the boundary cases explicitly. `sh:maxInclusive0.90` means 0.90 conforms and only a larger value fails. Likewise, a joint angle equal to either limit is valid, hence strict inequalities in the SPARQL filter. A surprising number of nearly correct submissions lose S3 points only because they use >=/<= in that filter.

4.5 Run and expected result

```
bash semantic/run_task1.sh --group <your-group-id>
# after Tasks 2 and 3:
bash semantic/run_task1.sh --group <your-group-id> --own
```

The command checks toolchains, prepares Jena, grounds the data, runs three validations, and scores the result. A correct implementation has no grounding structure violations, flags `target_mid` and `target_far` as out of range, leaves `target_near` clean, reports no joint-limit violation in the reference run, and matches all 30 records in `ta-answer-key.json`. The expected Task 1 score is 30/30.

TA/Tutor Note.

The 30-record probe contains 14 faulty and 16 clean records, including exact boundary values. Scoring is S1 structure 12 points, S2 expected problem detection 8 points, and S3 exact probe agreement 10 points. Inspect `output/ta-validation.ttl` for structure diagnostics and `output/probe-validation.ttl` for probe mismatches.

4.6 Assessment note: ontology documentation verification with Widoco

As part of the assessment of ontology completeness and readability, groups are encouraged to check whether their ontology can be successfully processed by Widoco [3], a tool for generating ontology documentation from OWL/RDF ontology files.

Widoco generation is not the only criterion for evaluating the group ontology. However, it will be used as one practical verification tool for checking whether the

submitted ontology is sufficiently well-formed, structurally complete, and readable for automatic documentation. If Widoco can generate documentation from your ontology without major errors, this is a good indication that the ontology file is suitable for further inspection and reuse.

A reference command is shown below (choose the release jar matching your JDK; the course environment's JDK 11 uses the `_JDK-11` build):

```
java -jar widoco-{{version-number}}-jar-with-dependencies_JDK-17.jar \
  -ontFile semantic/ontology/hw3-ontology.ttl \
  -outFolder docs/ontology-docs \
  -rewriteAll \
  -uniteSections
```

The output is a browsable HTML page (open `docs/ontology-docs/index-en.html`) that lists every class and property of the ontology in human-readable form – the same vocabulary you ground into and validate against in this task.

TA/Tutor Note.

Verified with Widoco v1.4.25 (`_JDK-11` build) on the course JDK 11 against `semantic/ontology/hw3-ontology.ttl`; the generated documentation for the reference ontology is kept under `hw3_demo/docs/ontology-docs/` as the comparison baseline. Treat Widoco as a well-formedness probe, not a score: a run that completes with only warnings (missing optional metadata such as authors or license) is acceptable; "major errors" means the ontology fails to parse or classes/properties are silently dropped from the documentation. When a group extends the ontology, check that their additions appear in the generated class/property index and carry `rdfs:label`/`rdfs:comment` annotations – Widoco makes missing annotation text immediately visible as empty documentation entries.

5 Task 2: Forward Kinematics (20 points)

Implement `your_fk(DH_params,q,base_pos)` in `fk.py`. Its inputs are the six classic D–H parameter records, six joint angles, and a base position. It returns the seven-dimensional end-effector pose $[x, y, z, q_x, q_y, q_z, q_w]$ and the 6×6 geometric Jacobian.

For each revolute joint, use the classic D–H transform

$$^{i-1}T_i = R_z(\theta_i)T_z(d_i)T_x(a_i)R_x(\alpha_i).$$

Save the preceding-frame origin p_{i-1} and z axis z_{i-1}. For end-effector position p_e, Jacobian column i is

$$J_i = \begin{bmatrix} z_{i-1} \times (p_e - p_{i-1}) \\ z_{i-1} \end{bmatrix}.$$

The D–H table in `get_ur5_DH_params()` is authoritative and intentionally differs slightly from the official UR5 specification. Do not use PyBullet or another library to directly compute FK/Jacobians, and do not modify the final `adjustment` block.

```
python fk.py              # visible easy / medium / hard tests
python fk.py -g -vp       # optional GUI and pose visualization
```

Success means zero FK and Jacobian error counts for every visible test file. The post-score semantic diagnosis should report conforming sampled records.

TA/Tutor Note.

Reference construction starts with the base transform, repeatedly multiplies the six classic D–H matrices, and stores origins/axes before each joint transform. The final pose quaternion is obtained from the accumulated rotation; the provided adjustment is then applied exactly as written. Hidden `ta1`/`ta2` cases are added only during grading. FK pose and Jacobian contribute 10 points each.
How to explain the Jacobian. The linear rows answer "which direction does the tool point move for a small rotation of this joint?"; the angular rows are the joint axis itself. The axis and origin must come from the frame *before* applying joint i. If every column is shifted, students usually stored frame i instead of frame $i-1$.

6 Task 3: Inverse Kinematics (40 points)

Implement `your_ik()` in `ik.py`. Warm-start from the current joint configuration, repeatedly call your own FK/Jacobian, and reduce a six-vector containing position and orientation error. A robust recommended update is damped least squares:

$$\Delta q = J^T \left(J J^T + \lambda^2 I \right)^{-1} \Delta x, \qquad q \leftarrow \mathrm{clip}(q + s\Delta q, q_{\min}, q_{\max}),$$

where λ is damping and s is the step rate. A rotation vector of $R_{\text{target}} R_{\text{current}}^T$ provides a useful orientation error. Stop when the error norm is below the threshold or the iteration budget is exhausted.

The implementation must not call PyBullet IK or another library's IK solver. `pybullet_ik()` is for behavioral comparison only. Grading invokes your function with default arguments, so tune the defaults rather than relying on command-line changes.

```
python ik.py
```

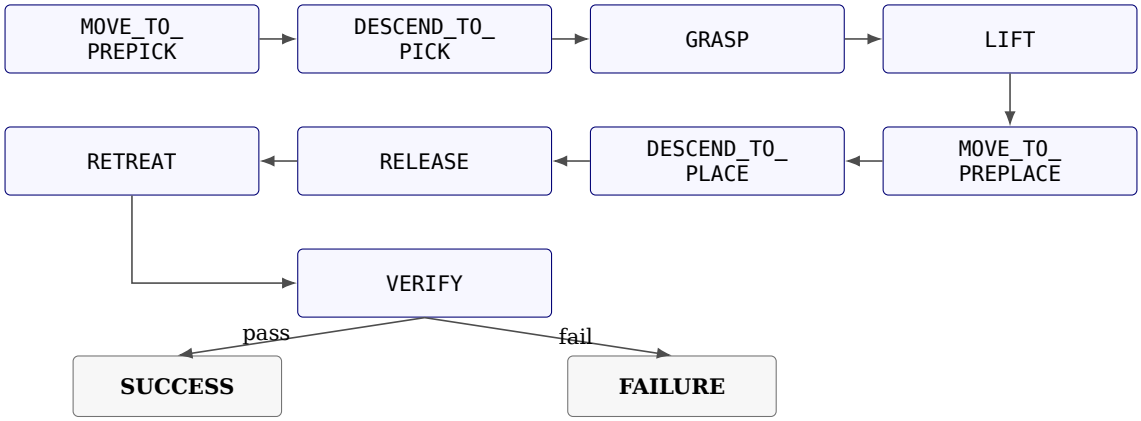
Aim for zero error counts on the easy, medium, and hard cases, mean positional error near 10^{-3} m, and conforming semantic diagnosis records.

TA/Tutor Note.

The verified reference uses damping $\lambda = 0.05$, step rate $s = 0.5$, the provided iteration/stop defaults, joint-limit clipping after every update, and `scipy.spatial.transform.Rotation` for the rotation-vector error. Common failures are quaternion subtraction, omitting orientation, using the wrong Jacobian row order, failing to warm-start, or returning unclipped joints.
Frame the tuning parameters by their effects: damping trades precision for stability near singularities; step rate controls how aggressively the solver moves; stopping threshold defines acceptable error; iteration budget limits runtime. Ask students to change one parameter at a time and record both mean error and failure count. A smaller damping value is not automatically better.

7 Task 4: FSM Manipulation Pipeline (10 points)

No code change is required. The provided FSM runs ten seeded pick-and-place episodes through



Every motion state solves the target using your IK and requires the achieved end-effector position to be within 3 cm. It also evaluates your FK on measured joint angles and requires agreement with the simulator within 1 cm. The pick descent continues until suction contact; verification requires the block to rest within 6 cm of the goal.

```
python fsm_task.py
python fsm_task.py -g       # optional PyBullet visualization
```

The expected result is ten `SUCCESS` episodes and 10/10 points. Failures name both the FSM state and reason, such as `IK_MISS`, `FK_MISMATCH`, `NO_CONTACT`, or `PLACE_MISS`; the semantic layer additionally reports applicable problem flags.

TA/Tutor Note.

The FSM is deterministic and protected. If it fails while unit tests pass, first inspect the named state, then distinguish IK tracking (3 cm), FK model agreement (1 cm), contact, and placement (6 cm). Do not award points for modifying `fsm_task.py` to bypass these checks.
Use the first failing state rather than the final `FAILURE` label as the primary clue. For example, `IK_MISSinMOVE_TO_PREPICK` points to target tracking before contact is relevant; `NO_CONTACT` after a successful pre-pick suggests approach height or descent behavior; `PLACE_MISS` with clean FK/IK checks suggests grasp/release timing rather than kinematic math.

8 Grading and Submission

Task 1: grounding/SHACL (S1 12 + S2 8 + S3 10)	30
Task 2: FK 10 + Jacobian 10	20
Task 3: IK hidden tests	40
Task 4: ten FSM episodes	10
Total	**100**

TA/Tutor Note.

Detailed deduction guide. Use the scores produced by the pristine grading scripts. Do not replace an automatic deduction with an impressionistic “code quality” deduction, and do not deduct twice for the same failed check. Always record the command, test file or episode, measured value, threshold, and points lost.

Task 1, S1 — grounding structure (12 points). Let n_{struct} be the number of valida-

tion results whose message begins with `STRUCTURE`. The score is

$$S_1 = \max(0, 12 - 2n_{\text{struct}}).$$

Consequently, each individual structure result deducts 2 points; six or more results reduce S1 to zero. Multiple results on one RDF node count separately because each describes a distinct failed constraint. Common causes include:

- missing `cora:Robot`, `soma:RevoluteJoint`, `soma:JointState`, or computation types;
- missing provenance, robot link, target link, status, residual, target distance, pose error, or Jacobian error;
- a robot or configuration linked to fewer or more than six members;
- a joint state without `isStateOfJoint` or `hasJointPosition`;
- a numeric literal serialized as `xsd:decimal`, integer, or string instead of the required `xsd:double`;
- a JSON/list string used where a structured pose or joint configuration node is required; or
- a wrong namespace or URI that prevents the TA shape from finding the expected class/property.

Use `output/ta-validation.ttl` as evidence. Quote the focus node and the complete `STRUCTURE:` message in feedback.

Task 1, S2 — expected problem detection (8 points). This part contains four independent binary checks worth 2 points each:

1. deduct 2 if `target_mid` is not flagged `ARM_OUT_OF_RANGE`;
2. deduct 2 if `target_far` is not flagged `ARM_OUT_OF_RANGE`;
3. deduct 2 if `target_near` has any non-structure problem flag; and
4. deduct 2 if any reference joint state is flagged `JOINT_LIMIT_VIOLATION`.

Typical causes are omitted/wrong target distance, an incorrect target URI, wrong grounding values, or a joint-limit rule that treats equality as illegal. Do not infer S2 from `sh:conforms`: the reference graph legitimately has problem results; evaluate the four named checks.

Task 1, S3 — student shapes against the 30-record probe (10 points). For each record i, the complete set of reported flag prefixes must equal the answer-key set K_i. A record is wrong for either a false negative, a false positive, or an incomplete flag set on a double-fault case. With 14 faulty and 16 clean records,

$$S_3 = 8\frac{n_{\text{faulty correct}}}{14} + 2\frac{n_{\text{clean correct}}}{16},$$

rounded by the scorer to one decimal place. Thus one faulty-record mismatch costs approximately 8/14 before rounding, while one clean-record mismatch costs 2/16. Frequent causes are:

- `sh:maxExclusive` instead of `sh:maxInclusive`, falsely flagging values exactly 0.90, 0.02, or 0.005;
- <=/>= in the SPARQL violation filter, falsely flagging a joint exactly at its bound;
- wrong target class/path, so a shape silently evaluates the wrong nodes;
- a message that does not begin with the required flag plus colon;
- testing only one side of the joint interval; or
- hard-coding an expected status string instead of validating the numeric residual, target distance, pose error, or joint angle.

The verified score gradients are useful sanity checks: correct shapes give 10.0; exclusive thresholds give about 9.8; a non-strict joint filter gives about 9.9; empty student shapes retain only the worked FK shape and give about 3.7. Use the actual scorer output as the final authority.

Task 2 — FK pose (10) and Jacobian (10). These are independent components. For test file f with N_f cases and F total test files, a pose case fails only when

$$\|\hat{p}_7 - p_7\|_2 > 0.005,$$

and a Jacobian case fails only when

$$\|\hat{J} - J\|_2 > 0.05.$$

Equality with a threshold passes. The implementation's per-failure decrement is

$$\delta_f = \frac{20/F}{0.3N_f},$$

applied separately to the pose and Jacobian component, with each file's component floored at zero. Therefore, a single case that fails both checks causes one pose deduction and one Jacobian deduction. Report both norms rather than saying only "FK incorrect." Typical root causes are wrong classic-D–H order, using modified D–H, applying the base offset twice, quaternion order or sign handling, collecting frame i rather than frame $i-1$ axes, swapping linear/angular Jacobian rows, or changing the protected adjustment block.
Using PyBullet or another direct FK/Jacobian API to produce the answer violates the task policy. Preserve the run output, identify the call, and escalate as an academic-integrity/policy case; do not invent an arbitrary numerical penalty.

Task 3 — inverse kinematics (40 points). For test file f, each target is executed and the achieved end-effector pose is measured. A case fails when its L2 error is strictly greater than 0.02 m. The per-failure decrement is

$$\delta_f^{\mathrm{IK}} = \frac{40/F}{0.3N_f},$$

with the score for each file floored at zero. Approximately 30% failed cases therefore exhaust that file's share. Grading calls `your_ik` with default arguments only. Deduct automatically for failures caused by poor default damping/step rate, insufficient iteration budget, incorrect orientation error, no warm start, no joint-limit clipping, singular updates, NaN/Inf output, wrong joint count, or a failure to return. Do not add a second manual deduction for the same numerical failures.
A crash originating in student IK code makes Task 3 unscorable and therefore 0/40 after reproducing it in the standard environment. Use of `calculateInverseKinematics` or another direct IK solver is a policy case to document and escalate.

Task 4 — FSM integration (10 points). There are ten deterministic episodes. Each successful episode earns one point:

$$S_4 = 10\frac{n_{\mathrm{success}}}{10} = n_{\mathrm{success}}.$$

An episode earns zero if any state terminates with an error. Preserve the first failure reason:

- `IK_MISS`: achieved versus commanded end-effector position is greater than 0.03 m;
- `FK_MISMATCH`: student FK versus simulator position is greater than 0.01 m;
- `NO_CONTACT`: descent reaches its floor without suction contact;
- `GRASP_FAILED`: no suction constraint is created; or
- `PLACE_MISS`: final block-to-goal XY error exceeds 0.06 m.

One failed episode loses exactly one point even if its log contains several diagnostic flags. `IK_MISS` and `FK_MISMATCH` are deterministic. For a contact/grasp/place failure that appears physically flaky, rerun once; count the failure only if reproducible. Never award partial episode credit for reaching an intermediate state.

Missing files, crashes, and protected-file changes.

- Reassemble only the four editable files in a pristine template before grading. Extra files alone are not a deduction.
- A missing required editable file or reproducible student-code crash gives zero for the task that cannot run, not automatically zero for other independently runnable tasks.
- Rule out a grading-machine or released-file problem before assigning zero. Save the first traceback as evidence.
- Ignore changes to protected files by grading in the pristine template. Deliberate score/threshold/test tampering is escalated, not handled through an improvised deduction.
- Semantic diagnosis printed after Tasks 2–4 is fail-soft and informational; a skipped diagnosis does not itself deduct from those task scores.
- No separate code-style score exists. The report has no fixed numerical deduction in the current automated 100-point allocation; missing-report, lateness, extension, or integrity decisions require instructor policy.

Feedback template. For every deduction, write:

> Task/component; command and case/episode; expected threshold or structure; observed value/message; points before and after; likely cause; exact file or report the student should inspect.

This makes deductions reproducible and gives students enough information to distinguish a modeling error, numerical error, integration error, and policy issue.

Submit:

1. `semantic/ground_execution.py`;
2. `semantic/shapes.ttl`;
3. `fk.py`;
4. `ik.py`; and
5. one PDF report following the required format below.

Required PDF report format

Name the file `<group-id>_hw3_report.pdf`. Begin with a cover block that lists the course, homework title, group ID, every member's student ID and name, and submission date. After the cover, use exactly the following four numbered top-level sections so that implementation, evidence, and analysis can be graded consistently.

1. **Semantic Grounding and SHACL Validation**
   - **1.1. Grounding design.** Explain how `robot_spec_to_triples()` and `ik_computation_to_triples()` map robot/execution data to structured RDF. Name the reused CORA, SOMA, POS, and QUDT terms, and explain why numerical values are typed `xsd:double` literals rather than JSON strings.
   - **1.2. SHACL constraints.** State the target class, property path, inclusive threshold, and required flag for arm range, convergence, and joint limits. Explain why the joint-limit rule requires SHACL–SPARQL.
   - **1.3. Execution evidence.** Insert one readable screenshot of the complete final output from `bashsemantic/run_task1.sh--group<group-id>`. The screenshot must show S1, all four S2 checks, S3 faulty/clean counts, and the 30-point total. Do not crop away the command or score summary.
   - **1.4. Validation analysis.** Summarize `output/validation.ttl`: identify the

focus nodes and flags, explain why `target_mid` and `target_far` are out of range, and explain why `target_near` conforms. Then summarize `output/probe-validation.ttl`: report faulty/clean agreement and discuss at least one threshold-boundary record and one joint-limit record. Distinguish false positives from false negatives if any mismatch remains.

2. Forward Kinematics and Geometric Jacobian

2.1. **Method.** Describe the classic D–H transform order, how the six transforms are accumulated, how the final pose and quaternion are obtained, and how the base position and protected adjustment are used.

2.2. **Jacobian derivation.** Include the formula $J_{v,i} = z_{i-1} \times (p_e - p_{i-1})$, $J_{\omega,i} = z_{i-1}$, and explain why preceding-frame axes/origins must be stored.

2.3. **Execution evidence.** Insert the complete final `pythonfk.py` score screenshot showing every test file, FK error counts, Jacobian error counts, total out of 20, and semantic diagnosis.

2.4. **Result analysis.** Provide a compact table with each test-file name, FK error count/score, and Jacobian error count/score. If any case fails, report its measured norm, threshold (0.005 or 0.05), likely cause, and attempted correction.

3. Inverse Kinematics

3.1. **Method.** Explain the iterative update, six-dimensional pose error, rotation-vector orientation error, damped least-squares equation, warm start, stopping condition, and joint-limit clipping.

3.2. **Parameters.** State the final default damping, step rate, maximum iterations, and stopping threshold. Explain how each affects convergence and why the selected values were used.

3.3. **Execution evidence.** Insert the complete final `pythonik.py` screenshot showing each test file, mean error, error count, score, total out of 40, and semantic diagnosis.

3.4. **Result and debugging analysis.** Tabulate each test file's mean error, error count, and score. Describe at least one difficulty encountered (for example oscillation, singularity, orientation error, or joint clipping), the evidence used to diagnose it, and the final fix.

4. FSM Manipulation Pipeline

4.1. **Pipeline explanation.** Explain the pick, grasp, transport, place, release, retreat, and verification stages. State where the student's IK controls motion and where FK is cross-checked against the simulator.

4.2. **Execution evidence.** Insert the complete final `pythonfsm_task.py` screenshot showing all ten episode results, total success count, score out of 10, and semantic diagnosis. Multiple screenshots are allowed if one image cannot remain legible.

4.3. **Result analysis.** Report the number of successes and failures and, for every failed episode, its first failed state and exact reason (`IK_MISS`, `FK_MISMATCH`, `NO_CONTACT`, `GRASP_FAILED`, or `PLACE_MISS`). Explain the relationship between numerical tolerances, semantic flags, and observed manipulation behavior.

4.4. **Conclusion.** Summarize whether the semantic layer, FK, IK, and FSM agree end to end. Identify one remaining limitation or concrete improvement even when all tests pass.

Formatting rules. Use selectable text for explanations; screenshots alone are not a report. Number and caption every figure and table, refer to each from the text, keep terminal text legible at normal zoom, include units on all numerical values, and do not paste entire source files. Short relevant snippets are acceptable when accompanied by an explanation. Every screenshot must come from the final submitted implementation and visibly include the command and result.

TA/Tutor Note.

Report completeness check. Mark each required subsection as present, partial, or absent. A screenshot is acceptable only if its command, individual results, and final score are readable. For Task 1, require an interpretation of both validation reports, not merely "30/30." For Tasks 2–3, require method and numerical analysis in addition to code images. For Task 4, require all ten episodes or an explicitly complete result table.
The automated scripts currently allocate all 100 numerical points. Therefore this checklist standardizes report acceptance and feedback but does not by itself authorize an extra point deduction. If the instructor adopts a report penalty or gate, apply it uniformly and announce the policy before grading.

Do not submit generated `semantic/output/`, `semantic/store/`, or `semantic/.cache/` directories.

TA/Tutor Note.

At grading time, copy only the four student-editable files into a clean template, add the hidden FK/IK test cases, and run all four commands. The TA reference material is under `hw3_demo/solutions/`; it must never be distributed. Check provenance options carefully: the released template uses `--group`, while the internal solved tree may use `--student-id`.

9 Troubleshooting Checklist

- For Task 1 structure errors, inspect `semantic/output/ta-validation.ttl`; verify types, cardinalities, and `xsd:double` literals.
- For probe mismatches, use inclusive thresholds and preserve the exact message prefixes.
- If Jena download fails, extract Jena 4.10.0 manually and set `JENA_HOME`.
- For FK failures, verify classic D–H order and collect preceding-frame axes before each transform.
- For unstable IK, increase damping, lower the step rate, use rotation-vector error, and clip joint limits.
- If GUI mode fails, run headless or verify the `DISPLAY` setting.

TA/Tutor Note.

TA debugging workflow. Diagnose in dependency order and stop at the first failing layer:

1. Confirm the correct environment: `which python`, `python --version`, then import NumPy, SciPy, and PyBullet.
2. Run Task 1 without `--own`. This isolates semantic work from the student's incomplete solvers.

3. Run FK before IK. IK depends directly on the pose and Jacobian returned by FK, so interpreting IK output while FK fails is usually misleading.
4. Run IK headless and inspect the first failing case, not only the mean.
5. Run the FSM last and use its first named state/reason together with the semantic flags.

Symptom-to-check map.

Symptom	First checks
S1 `STRUCTURE` violations	Required node type/property, exactly six joint states, typed `xsd:double`, correct `stu:/hw3:` URI.
S3 has only boundary mismatches	`sh:maxInclusive`; strict `</ >` for joint limits; exact message prefix.
FK pose wrong, Jacobian right-ish	Base translation, transform multiplication order, quaternion order (x, y, z, w), protected adjustment block.
FK pose right, Jacobian wrong	Store preceding-frame origins/axes; linear rows first and angular rows last; compute all columns using the final p_e.
IK diverges or oscillates	Verify FK first; reduce step rate; increase damping; check rotation-vector direction and units.
IK freezes at a joint bound	Inspect clipping and reachability; check whether the target is truly reachable before increasing iterations.
FSM `IK_MISS`	IK target/achieved positions, warm start, default parameters, and whether joint clipping prevents the requested pose.
FSM `FK_MISMATCH`	FK frame convention and adjustment; evaluate on the measured joint state rather than the commanded one.
Jena command unavailable	Java version, completed Jena cache, executable permissions, or a valid `JENA_HOME`.

TA FAQ.

Can a student use PyBullet FK or IK?
No. Simulator kinematics may be used only for comparison by provided code; the submitted implementations must derive their results independently.

Why does Task 1 run before FK/IK is implemented?
Its default records come from reference solvers. This separates semantic modeling errors from numerical solver errors and establishes the diagnostic contract first.

Does `--own` affect the Task 1 definition?
No. It changes the execution producer from reference solvers to the student's solvers; grounding structure and SHACL constraints remain the same.

Is a status string such as `SOLVED` sufficient?
No. Grading checks structured numerical evidence. The residual, target distance, pose error, joint configuration, and limits drive validation.

Why not subtract quaternions in IK?
Quaternion components are not a Euclidean orientation-error vector and q and $-q$ encode the same rotation. Use relative rotation followed by a rotation vector.

May students alter protected files to make the FSM pass?
No. Grade the four editable files in a clean template. Changes to thresholds, test cases, scoring code, diagnosis code, or the FSM do not form part of the solution.

What should be requested when a student asks for help?
Ask for the exact command, first traceback or first failed case, environment version, and the relevant validation report. A final total score alone is rarely enough to diagnose the cause.

References

[1] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. GitHub repository, 2021.

[2] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson Prentice Hall, 3 edition, 2005.

[3] Daniel Garijo. WIDOCO: A wizard for documenting ontologies. In *The Semantic Web – ISWC 2017*, volume 10588 of *Lecture Notes in Computer Science*, pages 94–102. Springer, 2017.

[4] Holger Knublauch and Dimitris Kontokostas. Shapes constraint language (shacl). W3c recommendation, World Wide Web Consortium, July 2017.